

ANSIBLE PLAYBOOK

Basics, Plays, Host Mapping
& Sequential Task Execution

Professional Reference Documentation

Version 1.0

Table of Contents

- 1. Introduction to Playbooks
- 2. YAML Syntax Essentials
- 3. Anatomy of a Playbook
 - 3.1 The Play
 - 3.2 The Task
- 4. Defining Plays
 - 4.1 Name and Hosts
 - 4.2 Privilege Escalation
 - 4.3 Variables and Facts
- 5. Mapping Target Hosts
 - 5.1 Host Patterns
 - 5.2 Group and Host Targeting
- 6. Tasks: The Unit of Work
 - 6.1 Module and Arguments
 - 6.2 Task Order and Dependencies
- 7. Sequential Execution
 - 7.1 Default Linear Strategy
 - 7.2 Handlers and Notification
 - 7.3 Includes and Imports
- 8. Complete Example Playbook
- 9. Best Practices
- 10. Quick Reference

1. Introduction to Playbooks

An Ansible playbook is a YAML file that describes a sequence of configuration steps, deployment actions, or orchestration tasks to be executed on a set of managed nodes. Playbooks are the primary unit of automation in Ansible and are designed to be human-readable, idempotent, and repeatable.

A playbook is composed of one or more **plays**. Each play maps a group of hosts to a list of **tasks**. Tasks are executed in order, one after another, for each host in the play. This deterministic execution model makes playbooks easy to reason about and debug.

Key characteristics:

- **Declarative** — you describe the desired end state, not the step-by-step commands to reach it.
- **Idempotent** — running the same playbook multiple times produces the same result without unintended changes.
- **Agentless** — no software is installed on managed nodes; Ansible pushes modules over SSH or WinRM.
- **Ordered** — tasks within a play execute sequentially by default, which provides predictable behaviour.

2. YAML Syntax Essentials

Playbooks are written in YAML (YAML Ain't Markup Language). YAML is a human-friendly data serialization format that uses indentation to represent structure. Strict adherence to syntax is required; a single misaligned space can cause a parsing error.

Fundamental rules:

- **Indentation** — use spaces, never tabs. Two spaces per level is the conventional standard.
- **Key-value pairs** — written as `key: value`. The colon must be followed by a space.
- **Lists** — items are prefixed with a hyphen and a space: `- item`.
- **Strings** — quotation marks are optional unless the value contains special characters or begins with a YAML indicator.
- **Comments** — begin with the hash symbol `#`.
- **Documents** — a playbook may contain multiple YAML documents separated by `---`, though a single document with a list of plays is more common.

Minimal valid YAML:

```
---
- name: Example play
  hosts: webservers
  tasks:
    - name: Ensure nginx is installed
      ansible.builtin.package:
        name: nginx
        state: present
```

3. Anatomy of a Playbook

A playbook is a YAML list. Each element of the list is a **play**. A play is a mapping that typically contains:

- `name` — a human-readable description of the play.
- `hosts` — the target hosts or groups from the inventory.
- `become` — whether to escalate privileges.
- `vars` — variables available to the play.
- `tasks` — an ordered list of tasks to execute.
- `handlers` — tasks triggered by notifications.
- `roles` — reusable collections of tasks, variables, and handlers.

3.1 The Play

A play is the bridge between hosts and tasks. It answers the questions: *Which hosts?* and *What should be done on them?* A playbook may contain multiple plays, allowing different configurations for different host groups within a single file.

3.2 The Task

A task is a single action that invokes an Ansible module with specific arguments. Tasks are the smallest unit of execution in a playbook. Each task should have a descriptive name so that output is understandable.

4. Defining Plays

4.1 Name and Hosts

Every play begins with a name and a hosts directive. The name is optional but strongly recommended. The hosts value is a host pattern that selects one or more inventory hosts or groups.

```
---
- name: Configure web servers
  hosts: webservers
  tasks:
    - name: Install nginx
      ansible.builtin.package:
        name: nginx
        state: present
```

4.2 Privilege Escalation

Many administrative tasks require root privileges. Use become to escalate. The default method is sudo, but this can be changed with become_method.

```
- name: Configure database servers
  hosts: databases
  become: true
  become_user: postgres
  tasks:
    - name: Ensure PostgreSQL is running
      ansible.builtin.service:
        name: postgresql
        state: started
```

4.3 Variables and Facts

Play-level variables are defined under the vars key. Facts are system properties gathered automatically by Ansible and can be used in tasks via the ansible_facts dictionary or the ansible_* variables.

```
- name: Configure application
  hosts: appservers
  vars:
    app_port: 8080
    app_user: deploy
  tasks:
    - name: Deploy configuration file
      ansible.builtin.template:
        src: app.conf.j2
        dest: /etc/app/app.conf
      vars:
        port: "{{ app_port }}"
        user: "{{ app_user }}"
```

5. Mapping Target Hosts

The `hosts` directive determines which managed nodes receive the tasks in a play. Ansible resolves this directive against the inventory to produce a list of target hosts.

5.1 Host Patterns

Ansible supports a rich pattern syntax:

Pattern	Meaning	Example
<code>all</code>	Every host in the inventory.	<code>hosts: all</code>
<code>*</code>	Same as <code>all</code> , but not recommended.	<code>hosts: "*"</code>
<code>group</code>	All hosts in a named group.	<code>hosts: webservers</code>
<code>host</code>	A single host.	<code>hosts: web01.example.com</code>
<code>group1:group2</code>	Union of two groups.	<code>hosts: webservers:databases</code>
<code>group1:!group2</code>	Intersection minus exclusion.	<code>hosts: all:!databases</code>
<code>group1:&group2;</code>	Intersection of two groups.	<code>hosts: webservers:&production;</code>
<code>regex</code>	Hosts matching a regular expression.	<code>hosts: ~web.*\.example\.com</code>

Table 1 — Common host pattern syntax.

5.2 Group and Host Targeting

Because inventory groups can be nested, targeting a parent group automatically includes all descendant hosts. This is useful for applying common configuration to an entire environment.

Example: apply a baseline configuration to all production hosts:

```
- name: Apply production baseline
  hosts: production
  become: true
  tasks:
    - name: Ensure NTP is installed
      ansible.builtin.package:
        name: chrony
        state: present
    - name: Ensure NTP is running
      ansible.builtin.service:
        name: chronyd
        state: started
        enabled: true
```

If `production` is a parent group with children `webservers`, `databases`, and `loadbalancers`, the play runs on every host in those child groups.

6. Tasks: The Unit of Work

6.1 Module and Arguments

A task calls a module and passes arguments to it. The module name is a key, and its value is a mapping of arguments. Many modules accept shorthand syntax, but the explicit mapping form is clearer and recommended.

```
- name: Ensure a user exists
  ansible.builtin.user:
    name: deploy
    state: present
    shell: /bin/bash
    groups: sudo
    append: true
```

The `ansible.builtin.` prefix refers to modules bundled with Ansible Core. Collections from Galaxy use a `namespace.collection.module` format.

6.2 Task Order and Dependencies

Tasks execute in the order they are written. If a task fails, subsequent tasks in the play are skipped for that host by default. To change this behaviour, use `ignore_errors` or `block/rescue`.

Common task directives:

- `name` — description shown in output.
- `when` — conditional execution.
- `loop` — iterate over a list.
- `register` — save the result for later use.
- `become` — task-level privilege escalation.
- `notify` — trigger a handler when the task reports a change.

Example with a loop and a condition:

```
- name: Install required packages
  ansible.builtin.package:
    name: "{{ item }}"
    state: present
  loop:
    - git
    - curl
    - vim
  when: ansible_os_family == "Debian"
```

7. Sequential Execution

7.1 Default Linear Strategy

Ansible uses the **linear** strategy by default. In this strategy, each task is executed on all target hosts before the next task begins. This is not the same as running all tasks on host 1, then all tasks on host 2. Instead, execution proceeds task by task across the entire host set.

Execution order for a play with two tasks and three hosts:

```
Task 1 -> host1
Task 1 -> host2
Task 1 -> host3
Task 2 -> host1
Task 2 -> host2
Task 2 -> host3
```

This order ensures that all hosts reach the same state before moving on, which simplifies reasoning about dependencies and failures.

7.2 Handlers and Notification

Handlers are special tasks that run only when notified by a task that reports a change. They are typically used to restart services after a configuration file has been modified. Handlers run once at the end of the play, in the order they are defined, regardless of how many tasks notify them.

```
- name: Configure nginx
  hosts: webservers
  become: true
  tasks:
    - name: Update nginx configuration
      ansible.builtin.template:
        src: nginx.conf.j2
        dest: /etc/nginx/nginx.conf
      notify: Restart nginx

  handlers:
    - name: Restart nginx
      ansible.builtin.service:
        name: nginx
        state: restarted
```

7.3 Includes and Imports

Large playbooks can be split into smaller files using `include_tasks` (dynamic) or `import_tasks` (static). Static imports are processed at parse time and preserve the top-to-bottom order of tasks. Dynamic includes are evaluated at runtime.

```
- name: Main playbook
  hosts: all
  tasks:
    - import_tasks: common.yml
    - import_tasks: webserver.yml
      when: "'webservers' in group_names"
```

8. Complete Example Playbook

The following playbook demonstrates multiple concepts covered in this document: plays, host targeting, variables, privilege escalation, tasks, loops, handlers, and sequential execution.

```
---
# site.yml -- deploy a simple web application

- name: Configure web servers
  hosts: webservers
  become: true
  vars:
    app_root: /var/www/html
    app_packages:
      - nginx
      - git
      - curl

  tasks:
    - name: Install required packages
      ansible.builtin.package:
        name: "{{ item }}"
        state: present
        loop: "{{ app_packages }}"

    - name: Ensure nginx is enabled and running
      ansible.builtin.service:
        name: nginx
        state: started
        enabled: true

    - name: Deploy application files
      ansible.builtin.git:
        repo: https://github.com/example/app.git
        dest: "{{ app_root }}"
        version: main
        notify: Restart nginx

    - name: Adjust file ownership
      ansible.builtin.file:
```

```
    path: "{{ app_root }}"
    owner: www-data
    group: www-data
    recurse: true

handlers:
  - name: Restart nginx
    ansible.builtin.service:
      name: nginx
      state: restarted

- name: Configure database servers
  hosts: databases
  become: true

tasks:
  - name: Ensure PostgreSQL is installed
    ansible.builtin.package:
      name: postgresql
      state: present

  - name: Ensure PostgreSQL is running
    ansible.builtin.service:
      name: postgresql
      state: started
      enabled: true
```

9. Best Practices

- **Name every play and task.** Descriptive names make output readable and failures easier to diagnose.
- **Use fully qualified module names.** Prefer `ansible.builtin.package` over `package` to avoid ambiguity.
- **Keep tasks idempotent.** Modules should describe a desired state, not execute arbitrary commands when a module exists.
- **Use variables, not hard-coded values.** Define variables at the appropriate level: `play`, `group_vars`, or `host_vars`.
- **Prefer handlers for service restarts.** Avoid restarting services unconditionally.
- **Split large playbooks.** Use roles or `import_tasks` to keep files focused and maintainable.
- **Validate with `--syntax-check` and `--check`.** Run `ansible-playbook --syntax-check` and `--check` before applying changes to production.
- **Version-control your playbooks.** Store them in Git alongside the inventory and roles.

10. Quick Reference

Command	Purpose
<code>ansible-playbook site.yml</code>	Execute a playbook.

Command	Purpose
<code>ansible-playbook site.yml --syntax-check</code>	Validate YAML and playbook syntax without running.
<code>ansible-playbook site.yml --check</code>	Dry run: report changes without applying them.
<code>ansible-playbook site.yml --diff</code>	Show file differences when changes occur.
<code>ansible-playbook site.yml --limit webservers</code>	Restrict execution to a group or host pattern.
<code>ansible-playbook site.yml --list-tasks</code>	List all tasks that would be executed.
<code>ansible-playbook site.yml --list-hosts</code>	List all hosts targeted by the playbook.
<code>ansible-playbook site.yml -v</code>	Increase verbosity (use <code>-vv</code> , <code>-vvv</code> for more detail).
<code>ansible-playbook site.yml --start-at-task "Task name"</code>	Begin execution at a specific task.
<code>ansible-playbook site.yml --step</code>	Prompt before each task (interactive stepping).

Table 2 — Frequently used playbook commands.

This document describes Ansible Core behaviour as of version 2.14 and later. Always consult the official documentation for the version in use.